How Does Chunking Work…

1. 1. Semantic Document Segmentation — Split at meaningful boundaries (headings, clauses, paragraphs) with priority ordering
2. 2. Context Management Across Chunks — Maintain document-level summary, defined terms, and party names across chunk boundaries
3. 3. Chunk-by-Chunk Agent Invocation — Sequential processing with 60s timeout, exponential backoff retries, and graceful failure handling
4. 4. Analysis Synthesis — Combine chunk results into a deduplicated comprehensive report via a final Bedrock Agent call
5. 5. Configurable Chunk Parameters — Tunable max/min chunk size, context window, and overlap with validation
6. 6. Small Document Pass-Through — Documents under 10K chars skip chunking entirely for fast processing
7. 7. Progress Feedback and Transparency — Real-time progress bar, elapsed time, and failure notifications
8. 8. File Size Limit Increase — Upload limit raised from 5MB to 25MB

The implementation plan breaks down into 10 groups with 28 sub-tasks following this order:

1) 1) Config & Models — Data classes, ChunkConfig with validation
2) 2) Chunking_Engine — Boundary detection, splitting, merging, overlap
3) 3) Checkpoint — Verify chunking tests pass
4) 4) Context_Manager — Summary generation, context window building, compression
5) 5) Synthesis_Engine — Deduplication, report assembly via agent call
6) 6) Checkpoint — Verify context/synthesis tests pass
7) 7) Document_Processor — Routing, retry logic, progress, file size validation
8) 8) Checkpoint — Verify processor tests pass
9) 9) Streamlit UI Integration — Sidebar controls, progress bar, report display, 25MB limit
10) 10) Final Checkpoint — End-to-end validation
11) 11) Each module has corresponding property-based tests using Hypothesis, and the task dependency graph enables parallel execution where possible.

To add HTTPS later (when you have a domain):

Create an ACM certificate for your domain (e.g., app.cloudage.llc)
Point your domain's CNAME to legal-doc-alb-552949067.eu-central-1.elb.amazonaws.com
Add an HTTPS listener: aws elbv2 create-listener --load-balancer-arn <ALB_ARN> --protocol HTTPS --port 443 --certificates CertificateArn=<ACM_CERT_ARN> --default-actions Type=forward,TargetGroupArn=<TG_ARN>
Redirect HTTP→HTTPS on the port 80 listener

Benefits of Multi-Agent Collaboration:

- • Quick setup: Create, deploy, and manage agents working together in minutes without complex coding
- • Composability: Integrate existing agents within a larger agent system
- • Efficient inter-agent communication: Consistent interface supporting parallel communication
- • Optimized collaboration modes: Choose between supervisor mode and supervisor with routing mode
- • Integrated trace and debug console: Visualize and analyze interactions between agents

Note: In internal benchmark testing, multi-agent collaboration has shown marked improvements compared to single-agent systems for handling complex, multi-step tasks.

The **Key Components** of this architecture include:

- • **Supervisor Agent:**
  - ◦ Receives and processes customer input
  - ◦ Orchestrates workflows across specialized agents
  - ◦ Ensures governance of agent interactions
  - ◦ Routes tasks to appropriate specialized agents
  - ◦ Synthesizes responses into coherent answers
  - ◦ Breaks down complex problems into manageable tasks
- • **Specialized Agents:**
  - ◦ Work within their domains of expertise
  - ◦ Process specific aspects of customer requests
  - ◦ Utilize specialized tools and knowledge bases
  - ◦ Return focused responses to the supervisor

The implementation process for creating a multi-agent collaboration system involves the following steps:

- • **Create and deploy specialized collaborator agents**, each configured to implement a specific task.
- • **Create a supervisor agent** or assign an existing agent the role of supervisor.
- • **Associate the alias versions** of the collaborator agents with the supervisor agent.
- • **Prepare and test** your multi-agent collaboration team.
- • **Deploy and invoke** the supervisor agent.

7 Review the multi-agent architecture components that you need to build in this project :
8
  - ◦ **Intent Classification Agent** - Analyzes and categorizes incoming requests.
  - ◦ **Tech Specialist Agent** - Handles technical support inquiries.
  - ◦ **Policy Specialist Agent** - Manages billing and policy-related questions.
  - ◦ **Supervisor Agent** - Orchestrates communication between specialized agents.

Note: Understand the orchestration flow: Customer request → Intent Classification → Appropriate Specialist Agent → Supervisor Agent → Final Response.

- • Deploying the multi agent collaboration architecture.
- • Creating and configuring specialized agents with distinct roles and capabilities.
- • Configuring communication between agents.
- • Testing the multi-agent collaboration system for complex scenarios.

For production applications, implement document chunking with semantic segmentation and context management to process larger documents efficiently while maintaining context across the entire document.

```
Use case : https://www.tamimi.com/client-services/sectors/
```